C prompt encoder + GRU generator

Task and source

The Phase 1 task is 128-word legal continuation given a 512-word prompt. The four official sources are Acts, High Court, Supreme Court, and Tribunal material. This report is a later reconstruction based on recovered source and archived run evidence, not an executed training notebook.

Architecture

Ten bidirectional Transformer blocks encode the prompt. A four-layer GRU decoder, initialized from mean prompt memory, generates in FP32 and cross-attends to encoded context; SwiGLU fusion and tied embeddings produce token logits.

Stage	512-word prompt -> Library-trained BPE -> 10-layer encoder
Continuation	4-layer FP32 GRU -> Cross-attn + fusion -> 128-word continuation

Token and optimization science

The model learns by minimizing next-token cross-entropy under causal masking. An attention head compares query/key vectors, obtains a weighted value mixture, and updates the residual stream; rotary phases give position-sensitive comparisons. Decoding then turns logits into a 128-word continuation. Tokenizer choice affects sequence length, vocabulary utilization, and the optimization budget. The original source config and scripts identify the actual optimizer, schedule, context and generation policy.

Reported outcome

0.8237 (historical Codabench screenshot). Historical screenshot numbers are not freshly retrieved leaderboard results. Archived local metric files, when present, provide separately measured BERTScore, ROUGE-L and BLEU-4; no metric is invented for runs without such evidence.

Reproduction boundary

The code package contains the recovered model, tokenizer, training, evaluation and export scripts, plus original small logs/metrics. The complete ZIP holds selected checkpoint weights and official data, which are too large or inappropriate to duplicate in GitHub. Re-execution requires their original hashes and path remapping.

Configured dimensions

The following values are read directly from the original configuration. A continued-training config may inherit the parent model graph; that relationship is identified in the README and source code.

Setting	Archived value
vocab_size	32768
max_seq_len	2048
max_prompt_len	8192
n_encoder_layers	10
n_gru_layers	4
d_model	768
n_heads	12
d_head	64
d_ff	2048
rope_theta	10000.0
dropout	0.0
gradient_checkpointing	True
pad_token_id	0

Archived evaluations

This table is a small sample of per-policy and per-checkpoint records, not a single comparable leaderboard ranking. Full machine-readable values and names are in evidence/archived_metric_index.json.

Metric file	BERTScore F1	ROUGE-L F1	BLEU-4
full/latest_policy0_dev.metrics.json	0.8205	0.2175	0.1042
full/latest_policy1_dev.metrics.json	0.8295	0.2160	0.1064
full/latest_policy2_dev.metrics.json	0.8335	0.2051	0.0988
full/latest_policy3_dev.metrics.json	0.8324	0.1969	0.0905
full/milestone_209914685_policy2_dev.metrics....	0.8288	0.1930	0.0723
full/milestone_209914685_policy3_dev.metrics....	0.8327	0.2127	0.1012
full/selected_confirmation.metrics.json	0.8338	0.1991	0.0939

Evidence count and caveat

Copied 47 compact log/metric/manifest artifacts from CS781_C_Complete_20261006.zip. They support the run lineage and measured local evaluations. A lower generation loss need not imply better BERTScore or legal accuracy; checkpoint and decoding policies must be evaluated separately. Original October 6 source archive contains no .ipynb file, so a later illustrated notebook is explicitly marked as reconstructed.